CO32: Fourier Optics

Bosco Kwong

November 2025

1 Abstract

This experiment analyses the Fourier transform of single slits, double slits, and 2-dimensional lattice. It shows how changing parameters in spatial domain, such as width, height and position, would affects the pattern in frequency domain.
For single slits, the fourier transform is a *sinc* function. For double slits, the fourier transform is a superposition of the diffraction envelope and interference fringes. Convolution Theorem is also verified by showing when $h(x) = f(x) * f(x)$, then $H(x) = [F(x)]^2$.

2 Introduction

Fourier transformation is a powerful mathematical tool to transform a function from spatial domain to frequency domain, which can be applied on various topics of physics. One of the main usage is analysing diffraction pattern, where the slits are in the spatial domain, and the pattern is in the frequency domain.
In this report, a few fundamental examples are investigated to see the relationship between the slits and the pattern. This report is structured as follows:

- Section 3 Methods: It details the equations and algorithms that have been used in this report to approximate the solution of each fourier transformation.
- Section 4 Single Slit: It starts with a regular diffraction pattern, then analyses the affects when shifting the position of the slit and changing its width and its height respectively.
- Section 5 Double Slit: It extends to a double slit scenario, where a diffraction envelope and small interference patterns are shown. It also shows the affect when changing the distance between the centres of the slits and others as in 4.
- Section 6 Convolution: It shows the self-convolution of a single slit is in fact a triangle. In addition, it verifies the Fourier transform of the self-convolution is the square of the Fourier transform of the original function $H(u) = [F(u)]^2$.

3 Methods

3.1 1-Dimensional Discrete Fourier Transform

The Fourier transform of a continuous function $f(x)$ is [1]

$$F(u) = \int_{-\infty}^{\infty} \mathrm{d}x\, f(x) e^{-2i\pi ux} \approx \sum_{x=-N}^{N-1} \left(f(x) e^{-\frac{i\pi ux}{N}} \right)$$

and the inverse is

$$F(u) = \int_{-\infty}^{\infty} \mathrm{d}u\, F(u) e^{2i\pi ux} \approx \sum_{x=-N}^{N-1} \left(f(x) e^{\frac{i\pi ux}{N}} \right)$$

3.2 Convolution

The convolution of two function $f(x)$ and $g(x)$ is [1]

$$h(x) = f(x) * g(x)$$

Then

$$H(u) = F(u)G(u)$$

Therefore $h(x)$ can be written as

$$\begin{aligned} h(x) &= FT^{-1}[F(u)G(u)] \\ &= FT^{-1}[FT(f) \times FT(g)] \end{aligned}$$

where $FT(f)$ is the Fourier transform of f and $FT^{-1}(f)$ is the inverse Fourier transform of f

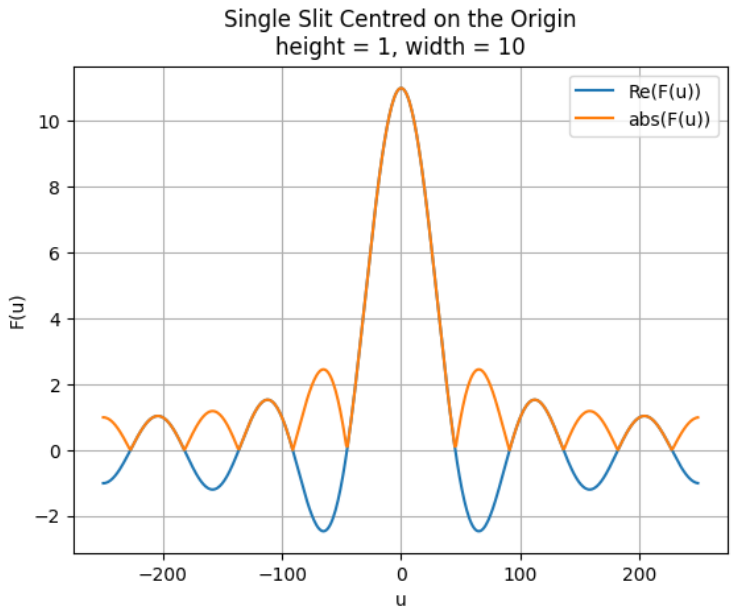
3.3 2-Dimensional Discrete Fourier Transform

The Fourier transform of a 2-dimensional continuous function $f(x, y)$ is [1]

$$\begin{aligned} F(u, v) &= \int_{-\infty}^{\infty} \mathrm{d}x \int_{-\infty}^{\infty} \mathrm{d}y\, f(x, y) e^{-i(ux+vy)} \\ &\approx \frac{1}{4NM} \sum_{x=-N}^{N-1} \sum_{y=-M}^{M-1} f(x, y) e^{-i\pi\left(\frac{ux}{N} + \frac{vy}{M}\right)} \quad \text{in a 2N×2M grid} \end{aligned}$$

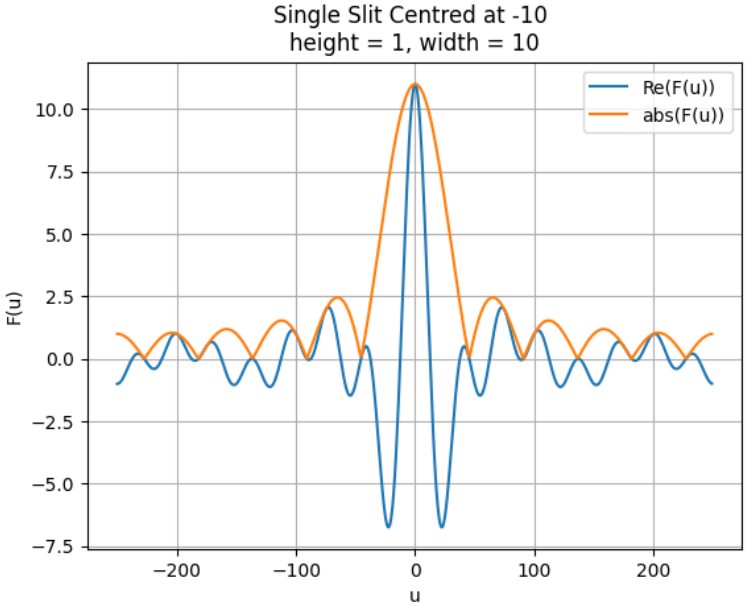
4 Single Slit

4.1 Centred on the Origin with Height 1 and Width 10

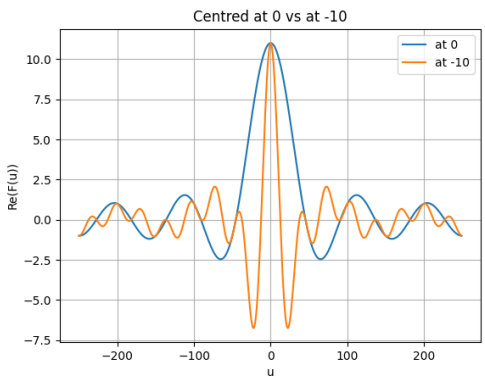


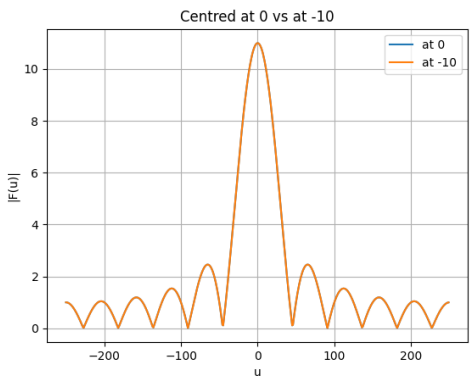
Because the function is an even function, the Fourier transform is real and it is a *sinc* function. Therefore, the magnitude is just the absolute of the real part. The peak amplitude is 11, which is the zero frequency component.

4.2 Centred at -10 with Height 1 and Width 10



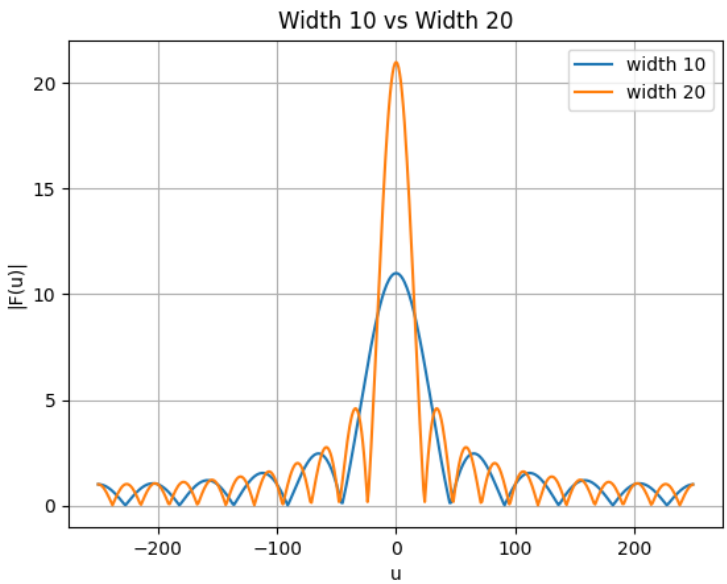
The function is no longer odd or even function, therefore the Fourier transform consists of both real and imaginary parts, then the magnitude is not just the absolute of the real part.





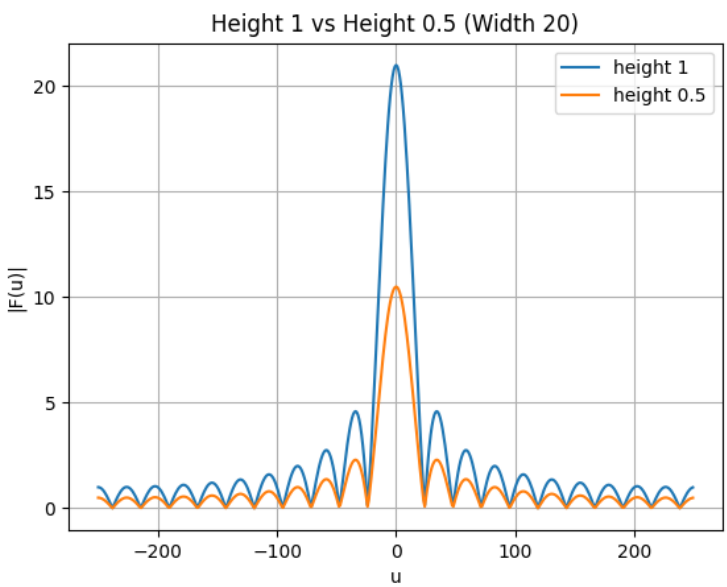
The magnitudes are identical, as according to Fourier Shift Theorem, a shift in spatial domain only adds a linear phase shift to the transform, but does not change the magnitude at all. Therefore, the real part is different because of the phase change after using Euler's formula $e^{ix} = \cos x + i \sin x$.

4.3 Comparison of Width 10 and Width 20



The peak amplitude is now 21 instead of 11, which is approximately double. It is because the zero frequency component is just the sum of the input. The width in frequency domain is inversely proportional to the width in spatial domain. This phenomenon leads to the Heisenberg Uncertainty Principle in quantum mechanics.

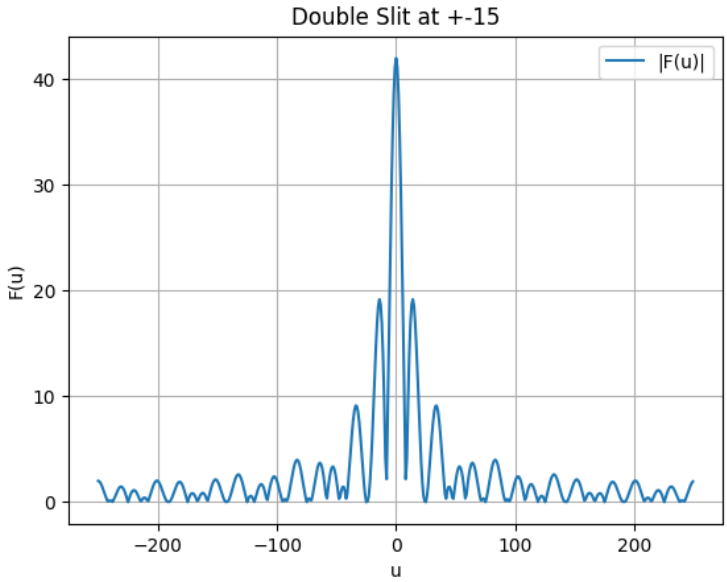
4.4 Comparison of Height 1 and Height 0.5



The peak amplitude is now $21 \times \frac{1}{2} = 10.5$ instead of 21, which is the same ratio. As changing the amplitude in spatial domain changes the amplitude in frequency domain proportionally, while everything else stays the same.

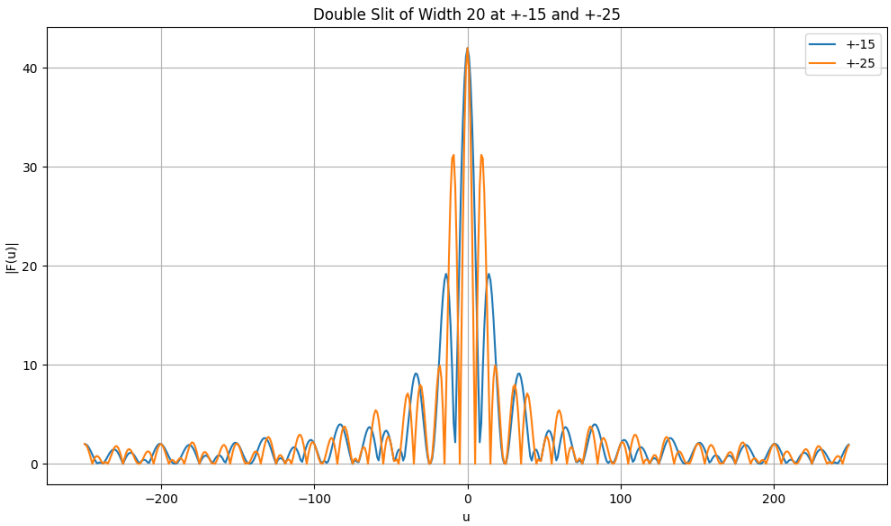
5 Double Slit

5.1 Centred at -15 and 15 with Width 20 Each



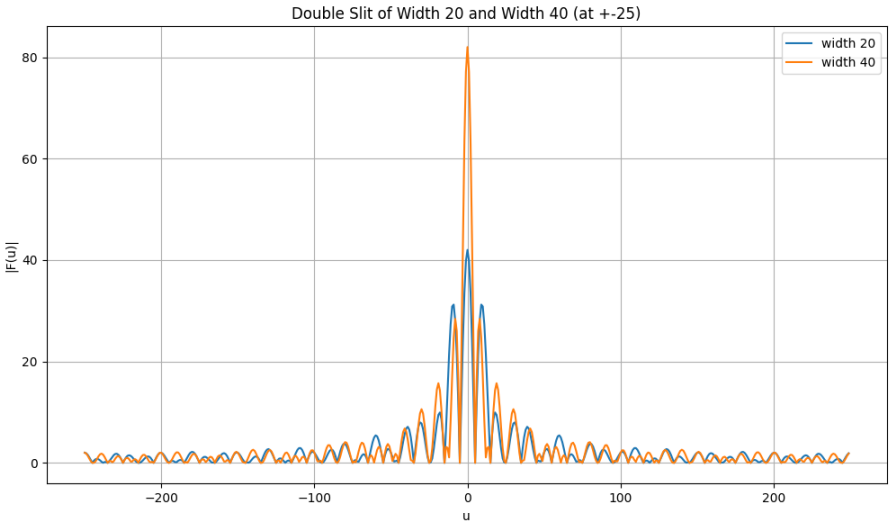
There is a diffraction *sinc* function envelope, with the first minimum at $u \approx \pm 50$, with many small oscillations in between. Each small oscillation is a fringe, which is caused by two slits interfering with each other.

5.2 Comparison of Centred at -25 and 25 with Width 20 Each



The diffraction envelopes are the same because their widths of the slits are the same. However, the interference fringers for slits at ± 25 are closer than at ± 15. It is because the spacing is inversly prortional to the distance bewteen slits.

5.3 Comparison of Width 20 and Width 40



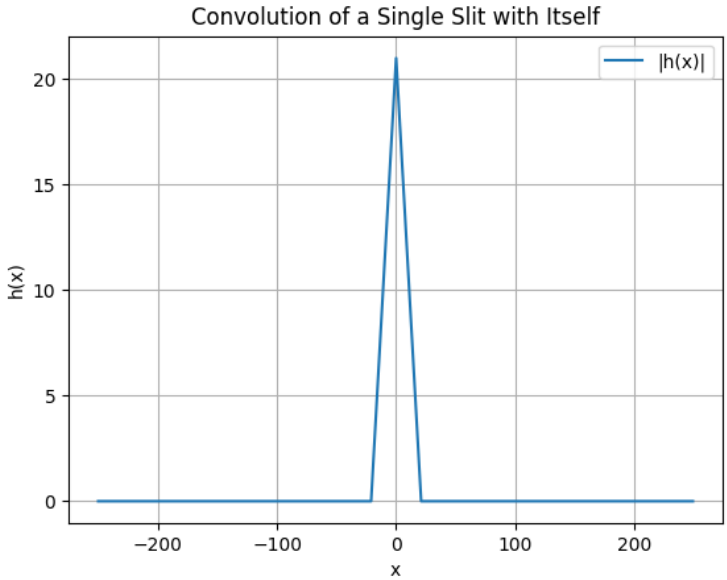
The fringe spacings are the same as it only depends on the distance between the centres of slits, which is the same. The first minimum is at $u \approx \pm 25$. It is because the slit is twice as wide now, the diffraction envelope is twice as narrow due to inversly proportionality. The peak amplitude is now 82 instead of 42, which is approximately doubled.

5.4 Comparison of Height 1 and Height 0.5

The diffraction envelope and interference fringes are exactly the same. The only difference is all amplitudes are exactly half of the original, which is 41 instead of 82.

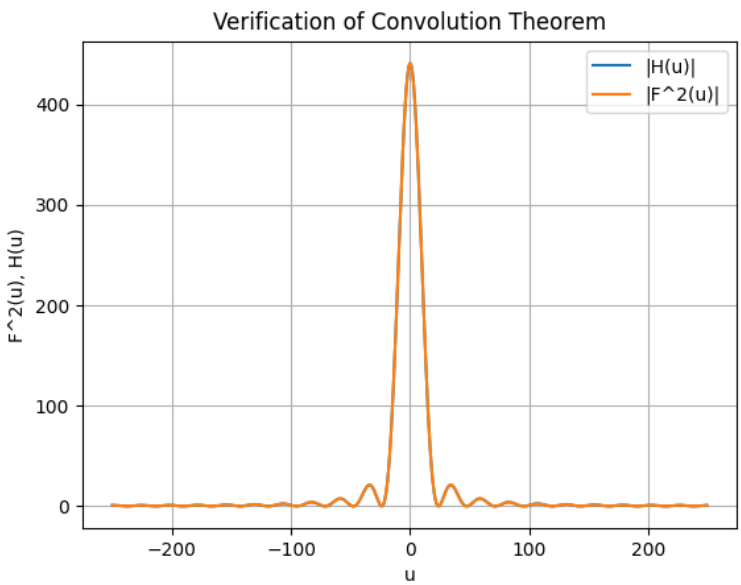
6 Convolution

6.1 Single Slit with Itself



The convolution $h(x)$ of 2 straight lines $f(x)$ (from a single slit of width 20) is a triangle as expected. The peak of a self-convolution is the area under the original function, which is 21.

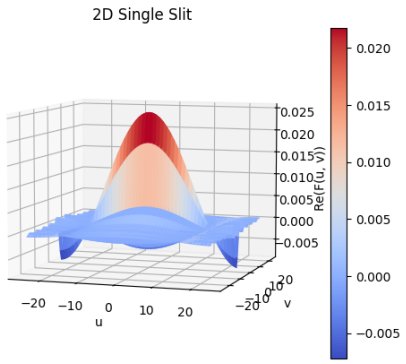
6.2 Verification of the Convolution Theorem

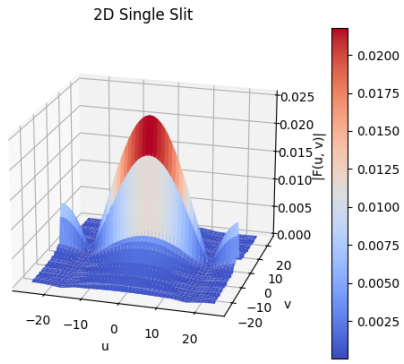


The graphs of $H(u)$ and $[F(u)]^2$ are exactly identical, and the peak is at $21^2 = 441$ as expected. Thus, it verifies the convolution theorem $H(u) = [F(u)]^2 \quad \text{where} \quad h(x) = f(x) * f(x)$.

7 2-Dimensional Lattice

7.1 Problem C: Single Slit of Width 3 and Length 20



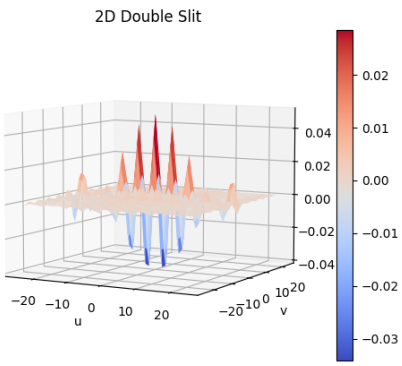


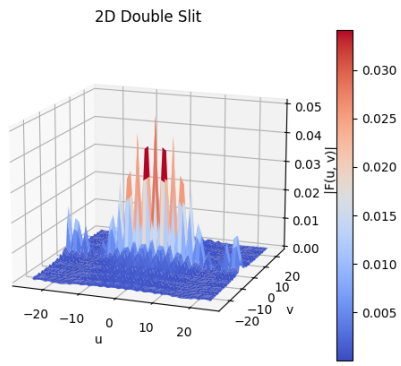
Because the function is even in both x and y direction, the Fourier transform is real and a *sinc* function in both u and v direction. Therefore, the magnitude is just the absolute of the real part. As mentioned in section 4, the width in frequency domain is inversely proportional to the width in spatial domain. The width in x direction is 3, which is much less than 20, the width in y direction. Thus, the diffraction in u direction is much wider than in v direction.

The peak value is given by the sum of the input divided by the normalisation factor, which is

$$3 \times 21 \times 1 \times \frac{1}{4 \times 25 \times 25} = 0.0252$$

7.2 Problem D: Double Slit of Width 3 and Length 20 at +-10





The Fourier transform is also real because of the even function. The diffraction pattern like above becomes diffraction envelopes in u and v direction, and it is wider in u direction. Because the separation of the slit is in x direction, there are interference fringes in u direction only, but not in v direction.

The peak value is now twice as high as above because the double slits consist of 2 identical single slit from section 7.1. Therefore the peak value is $2 \times 0.0252 = 0.0504$.

8 Conclusion

This experiment analyses the Fourier transforms of fundamental slits in both 1-dimension and 2-dimension, showing the relationships between spatial domain and frequency domain, which can be used to predict the diffraction and interference pattern.

In section 4, the experiment shows that the Fourier transform of a single slit is a *sinc* function, and show 4 important properties by varying the parameters of the slit. First, shifting the slit's position does nothing to the magnitude of the pattern, it only applies a phase shift, which can be seen in the real and imaginary component. Then, the width of the diffraction pattern in the frequency domain is inversely proportional to the width of the slit in spatial domain. Furthermore, the amplitude of the diffraction pattern is proportional to the height of the slit. Lastly, the peak amplitude, which is the zero frequency, is equals to the sum of the input slits (sometimes with a normalisation factor).

In section 5, the experiment shows that the pattern is a superposition of a diffraction envelope and interference fringes. The properties of the diffraction pattern is determined by the dimension of a single slit, while the propoerties of interference fringes are determined by the distance bewteen the centres of the slits. Furthermore, the experiment shows that the spacing of each interference fringe is inversely proportional to the separation distance of the slits.

In section 6, the convolution theorem is verfied by the experiment. The convolution of a single slit with itself is a traingle as expected, where the Fourier transform of the triangle is equals to the square of the Fourier transform of the slit, confirming the equation $H(u) = [F(u)]^2$.

In section 7, the experiment extends these principles into 2-dimensional situations, and shows that the properties are the same as in 1-dimension, where x and y directions in spatial domain correspond to u and v in frequency domain respectively. The peak value is still equals to the sum of the input slits with a normalisation factor.

References

[1] Lab scripts (2024) Lab Scripts, Oxford Physics Practical Course. Available at: https://www-teaching.physics.ox.ac.uk/practical_course/scripts/ (Accessed: 08 November 2025).

Appendix

main.py

```python
from dft_3a import single_slit
from dft_3b import double_slit
from dft_3c import convolution
from dft_4 import slit_2d
from matplotlib import pyplot as plt

def main():
    dft = single_slit(250)
    dft.centre_width10(real=True)
    plt.show()

if __name__ == "__main__":
    main()
```

```python
import numpy as np
import numpy.typing as npt

class fourier:
    def __init__(self):
        pass

    def ft1(self, X:npt.NDArray[np.complexfloating])->npt.NDArray[np.complexfloating]:
        """
        Author: Bosco Kwong
        Date: 8/11/2025
        Summary:
            Fourier transform function for a 1-dimensional signal

        Args:
            X (npt.NDArray[np.complexfloating]): a column vector of signal with N complex elements

        Returns:
            npt.NDArray[np.complexfloating]: the Fourier transfored signal in the form of a column vector of N complex elements.

        Example use:
            t = np.arange(-N, N)
            x = np.array([1 if -10<=i<=10 else 0 for i in t])
            y = fourier().ft1(x)
        """
        X_size: int = X.size
        N: int= X_size // 2
        t: npt.NDArray[np.integer] = np.arange(-N, N)

        y: npt.NDArray[np.complexfloating] = np.zeros(X_size, dtype=np.complexfloating)
        for i, u in enumerate(t):
            y[i] = sum(f * np.exp(-np.pi*1j*x*u/N) for x, f in zip(t, X))
        return y

    def inverse_ft1(self, X:npt.NDArray[np.complexfloating])->npt.NDArray[np.complexfloating]:
        """
        Author: Bosco Kwong
        Date: 8/11/2025
        Summary:
            Inverse Fourier transform function for a 1-dimensional signal

        Args:
            X (npt.NDArray[np.complexfloating]): a column vector of signal with N complex elements

        Returns:
            npt.NDArray[np.complexfloating]: the inverse Fourier transfored signal in the form of a column vector of N complex elements.
```

```python

        Example use:
            t = np.arange(-N, N)
            x = np.array([1 if -10<=i<=10 else 0 for i in t])
            y = fourier().inverse_ft1(x)
        """
        X_size: int = X.size
        N: int= X_size // 2
        t: npt.NDArray[np.integer] = np.arange(-N, N)

        y: npt.NDArray[np.complexfloating] = np.zeros(X_size, dtype=np.complexfloating)
        for i, u in enumerate(t):
            y[i] = sum(f * np.exp(np.pi*1j*x*u/N) for x, f in zip(t, X))
        return y

    def convolution1(self, u:npt.NDArray[np.complexfloating], v:npt.NDArray[np.complexfloating]) -> npt.NDArray[np.complexfloating]:
        return self.inverse_ft1(self.ft1(u)*self.ft1(v)) / u.size

    def ft2(self, X:npt.NDArray[np.complexfloating])->npt.NDArray[np.complexfloating]:
        """
        Author: Bosco Kwong
        Date: 8/11/2025
        Fourier transform function for 2-dimensional signal.

        Args:
            X (npt.NDArray[np.complexfloating]): a matrix of 2D signal with the size (2M x 2N).

        Returns:
            npt.NDArray[np.complexfloating]:the Fourier transformed signal in the form of a (2M x 2N) matrix with complex elements.

        Example use:
            x = np.zeros((2*self.__N, 2*self.__M), dtype=np.complex128)
            x[10][10]=1
            y = fourier().ft2(x)
        """
        N: int
        M: int
        row, column = X.shape
        N = row // 2
        M = column // 2
        tn: npt.NDArray[np.integer] = np.arange(-N, N)
        tm: npt.NDArray[np.integer] = np.arange(-M, M)
        Y: npt.NDArray[np.complexfloating] = np.zeros(X.shape, dtype=np.complexfloating)

        for i, u in enumerate(tn):
            for j, v in enumerate(tm):
                Y[i][j] = sum(X[x+N][y+M] * np.exp(-np.pi*1j*(u*x/N+v*y/M)) for x in tn for y in tm)
        Y /= (4*N*M)
        return Y
```

```python
import numpy as np
from fourier import fourier
from matplotlib import pyplot as plt

class single_slit:
    def __init__(self, N):
        self.__N = N
        self.__t = np.arange(-N, N)

    def centre_width10(self, **kwargs):
        fx = kwargs.get("fx", False)
        real = kwargs.get("real", False)
        imag = kwargs.get("imag", False)
        mag = kwargs.get("mag", False)

        t = self.__t
        x = np.array([1 if -5<=i<=5 else 0 for i in t])
        y = fourier().ft1(x)

        if fx: plt.plot(t, x, label="x")
        if real: plt.plot(t, y.real, label="Re(F(u))")
        if imag: plt.plot(t, y.imag, label="Im(F(u))")
        if mag: plt.plot(t, np.abs(y), label="|F(u)|")
        plt.xlabel("u")
        plt.ylabel("F(u)")
        plt.legend()
        plt.grid(True)

    def shifted_width10(self, **kwargs):
        fx = kwargs.get("fx", False)
        real = kwargs.get("real", False)
        imag = kwargs.get("imag", False)
        mag = kwargs.get("mag", False)

        t = self.__t
        x = np.array([1 if -5-10<=i<=5-10 else 0 for i in t])
        y = fourier().ft1(x)

        if fx: plt.plot(t, x, label="x")
        if real: plt.plot(t, y.real, label="Re(F(u))")
        if imag: plt.plot(t, y.imag, label="Im(F(u))")
        if mag: plt.plot(t, np.abs(y), label="|F(u)|")
        plt.xlabel("u")
        plt.ylabel("F(u)")
        plt.legend()
        plt.grid(True)

    def centre_width20(self, **kwargs):
        fx = kwargs.get("fx", False)
        real = kwargs.get("real", False)
        imag = kwargs.get("imag", False)
        mag = kwargs.get("mag", False)

        t = self.__t
        x = np.array([1 if -10<=i<=10 else 0 for i in t])
        y = fourier().ft1(x)

```

```
        if fx: plt.plot(t, x, label="x")
        if real: plt.plot(t, y.real, label="Re(F(u))")
        if imag: plt.plot(t, y.imag, label="Im(F(u))")
        if mag: plt.plot(t, np.abs(y), label="|F(u)|")
        plt.ylabel("F(u)")
        plt.legend()
        plt.grid(True)

    def centre_width20_heighthalf(self, **kwargs):
        fx = kwargs.get("fx", False)
        real = kwargs.get("real", False)
        imag = kwargs.get("imag", False)
        mag = kwargs.get("mag", False)

        t = self.__t
        x = np.array([.5 if -10<=i<=10 else 0 for i in t])
        y = fourier().ft1(x)

        if fx: plt.plot(t, x, label="x")
        if real: plt.plot(t, y.real, label="Re(F(u))")
        if imag: plt.plot(t, y.imag, label="Im(F(u))")
        if mag: plt.plot(t, np.abs(y), label="|F(u)|")
        plt.xlabel("u")
        plt.ylabel("F(u)")
        plt.legend()
        plt.grid(True)

if __name__ == "__main__":
    dft = single_slit(250)
    #dft.centre_width10(real=False, mag=True)
    #dft.shifted_width10(real=False, mag=True)
    dft.centre_width20(real=False, mag=True)
    dft.centre_width20_heighthalf(mag=True)
    plt.title("Height 1 vs Height 0.5 (Width 20)")
    plt.ylabel("|F(u)|")
    plt.show()
```

```python
import numpy as np
from fourier import fourier
from matplotlib import pyplot as plt

class double_slit:
    def __init__(self, N):
        self.__N = N
        self.__t = np.arange(-N, N)

    def at_pm15(self, **kwargs):
        fx = kwargs.get("fx", False)
        real = kwargs.get("real", False)
        imag = kwargs.get("imag", False)
        mag = kwargs.get("mag", False)

        t = self.__t
        x = np.array([1 if -15-10<=i<=-15+10 or 15-10<=i<=15+10 else 0
            for i in t])
        y = fourier().ft1(x)

        if fx: plt.plot(t, x, label="x")
        if real: plt.plot(t, y.real, label="Re(F(u))")
        if imag: plt.plot(t, y.imag, label="Im(F(u))")
        if mag: plt.plot(t, np.abs(y), label="|F(u)|")
        plt.xlabel("u")
        plt.ylabel("F(u)")
        plt.legend()
        plt.grid(True)

    def at_pm25(self, **kwargs):
        fx = kwargs.get("fx", False)
        real = kwargs.get("real", False)
        imag = kwargs.get("imag", False)
        mag = kwargs.get("mag", False)

        t = self.__t
        x = np.array([1 if -25-10<=i<=-25+10 or 25-10<=i<=25+10 else 0
            for i in t])
        y = fourier().ft1(x)

        if fx: plt.plot(t, x, label="x")
        if real: plt.plot(t, y.real, label="Re(F(u))")
        if imag: plt.plot(t, y.imag, label="Im(F(u))")
        if mag: plt.plot(t, np.abs(y), label="|F(u)|")
        plt.xlabel("u")
        plt.ylabel("F(u)")
        plt.legend()
        plt.grid(True)

    def at_pm25_doublewidth(self, **kwargs):
        fx = kwargs.get("fx", False)
        real = kwargs.get("real", False)
        imag = kwargs.get("imag", False)
        mag = kwargs.get("mag", False)

        t = self.__t
        x = np.array([1 if -25-20<=i<=-25+20 or 25-20<=i<=25+20 else 0
```

```
            for i in t])
        y = fourier().ft1(x)

        if fx: plt.plot(t, x, label="x")
        if real: plt.plot(t, y.real, label="Re(F(u))")
        if imag: plt.plot(t, y.imag, label="Im(F(u))")
        if mag: plt.plot(t, np.abs(y), label="|F(u)|")
        plt.xlabel("u")
        plt.ylabel("F(u)")
        plt.legend()
        plt.grid(True)

    def at_pm25_halfheight(self, **kwargs):
        fx = kwargs.get("fx", False)
        real = kwargs.get("real", False)
        imag = kwargs.get("imag", False)
        mag = kwargs.get("mag", False)

        t = self.__t
        x = np.array([.5 if -25-10<=i<=-25+10 or 25-10<=i<=25+10 else 0
            for i in t])
        y = fourier().ft1(x)

        if fx: plt.plot(t, x, label="x")
        if real: plt.plot(t, y.real, label="Re(F(u))")
        if imag: plt.plot(t, y.imag, label="Im(F(u))")
        if mag: plt.plot(t, np.abs(y), label="|F(u)|")
        plt.xlabel("u")
        plt.ylabel("F(u)")
        plt.legend()
        plt.grid(True)

if __name__ == "__main__":
    dft = double_slit(250)
    dft.at_pm15(fx=False, real=False, mag=True)
    #dft.at_pm25(fx=False, real=False, mag=True)
    #dft.at_pm25_doublewidth(fx=False, real=False, mag=True)
    #dft.at_pm25_halfheight(mag=True)
    plt.title("Double Slit at +-15")
    plt.show()
```

```python
import numpy as np
from fourier import fourier
from matplotlib import pyplot as plt
from dft_3a import single_slit

class convolution:
    def __init__(self, N):
        self.__N = N
        self.__t = np.arange(-N, N)

    def singleslit(self, **kwargs):
        fx = kwargs.get("fx", False)
        real = kwargs.get("real", False)
        imag = kwargs.get("imag", False)
        mag = kwargs.get("mag", False)

        t = self.__t
        x = np.array([1 if -10<=i<=10 else 0 for i in t])
        y = fourier().convolution1(x, x)

        if fx: plt.plot(t, x, label="x")
        if real: plt.plot(t, y.real, label="Re(h(x))")
        if imag: plt.plot(t, y.imag, label="Im(h(x))")
        if mag: plt.plot(t, np.abs(y), label="|h(x)|")
        plt.xlabel("x")
        plt.ylabel("h(x)")
        plt.legend()
        plt.grid(True)

    def singleslit_check(self, **kwargs):
        fx = kwargs.get("fx", False)
        real = kwargs.get("real", False)
        imag = kwargs.get("imag", False)
        mag = kwargs.get("mag", False)

        t = self.__t
        x = np.array([1 if -10<=i<=10 else 0 for i in t])
        h = fourier().convolution1(x, x)
        H = fourier().ft1(h)
        F2 = (fourier().ft1(x))**2

        if fx:
            plt.plot(t, x, label="x")
            plt.plot(t, h, label="h")
        if real:
            plt.plot(t, H.real, label="Re(H(u))")
            plt.plot(t, F2.real, label="Re(F^2(u))")
        if imag:
            plt.plot(t, H.imag, label="Im(H(u))")
            plt.plot(t, F2.imag, label="Im(F^2(u))")
        if mag:
            plt.plot(t, np.abs(H), label="|H(u)|")
            plt.plot(t, np.abs(F2), label="|F^2(u)|")
        plt.xlabel("u")
        plt.ylabel("F^2(u), H(u)")
        plt.legend()
        plt.grid(True)
```

```python

if __name__ == "__main__":
    con = convolution(250)
    #con.singleslit(mag=True)
    con.singleslit_check(mag=True)
    plt.title("Verification of Convolution Theorem")
    plt.show()
```

```python
import numpy as np
import numpy.typing as npt
from fourier import fourier
from matplotlib import pyplot as plt
from matplotlib import cm

class slit_2d:
    def __init__(self, N2, M2):
        self.__N: int = N2 // 2
        self.__M: int = M2 // 2
        self.__tn: npt.NDArray[np.integer] = np.arange(-self.__N, self.__N)
        self.__tm: npt.NDArray[np.integer] = np.arange(-self.__M, self.__M)

    def singleslit(self, **kwargs):
        fxy = kwargs.get("fx", False)
        real = kwargs.get("real", False)
        imag = kwargs.get("imag", False)
        mag = kwargs.get("mag", False)

        tn = self.__tn
        tm = self.__tm
        x = np.zeros((2*self.__N, 2*self.__M), dtype=np.complex128)
        x[np.ix_([i+self.__N for i in tn if -1 <= i <= 1], [j+self.__M for j in tm if -10 <= j <= 10])] = 1
        y = fourier().ft2(x)

        fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
        U, V = np.meshgrid(tn, tm, indexing="ij")
        if fxy: surf = ax.plot_surface(U, V, x, cmap=cm.coolwarm)
        if real: surf =  ax.plot_surface(U, V, y.real, cmap=cm.coolwarm)
        if imag: surf = ax.plot_surface(U, V, y.imag, cmap=cm.coolwarm)
        if mag: surf = ax.plot_surface(U, V, np.abs(y), cmap=cm.coolwarm)
        ax.set_title("2D Single Slit")
        ax.set_xlabel('u')
        ax.set_ylabel('v')
        if real: ax.set_zlabel("Re(F(u, v))")
        if mag: ax.set_zlabel("|F(u, v)|")
        fig.colorbar(surf)


    def doubleslit(self, **kwargs):
        fxy = kwargs.get("fx", False)
        real = kwargs.get("real", False)
        imag = kwargs.get("imag", False)
        mag = kwargs.get("mag", False)

        tn = self.__tn
        tm = self.__tm
        x = np.zeros((2*self.__N, 2*self.__M), dtype=np.complex128)
        x[np.ix_([i+self.__N for i in tn if (-11 <= i <= -9) or (9 <= i <= 11)], [j+self.__M for j in tm if -10 <= j <= 10])] = 1
        y = fourier().ft2(x)

```

```
        fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
        U, V = np.meshgrid(tn, tm, indexing="ij")
        if fxy: surf = ax.plot_surface(U, V, x, cmap=cm.coolwarm)
        if real: surf = ax.plot_surface(U, V, y.real, cmap=cm.coolwarm)
        if imag: surf = ax.plot_surface(U, V, y.imag, cmap=cm.coolwarm)
        if mag: surf = ax.plot_surface(U, V, np.abs(y), cmap=cm.
            coolwarm)
        ax.set_title("2D Double Slit")
        ax.set_xlabel('u')
        ax.set_ylabel('v')
        if real: ax.set_zlabel("Re(F(u, v))")
        if mag: ax.set_zlabel("|F(u, v)|")
        fig.colorbar(surf)

if __name__ == "__main__":
    dft = slit_2d(50, 50)
    #dft.singleslit(real=False, mag=True)
    dft.doubleslit(real=False, mag=True)

    plt.show()
```